

# **Retrieval-augmented generation (RAG)**

**194.093 Natural Language Processing and Information Extraction  
2025WS**

**Lecture 9, 12/03/2025**

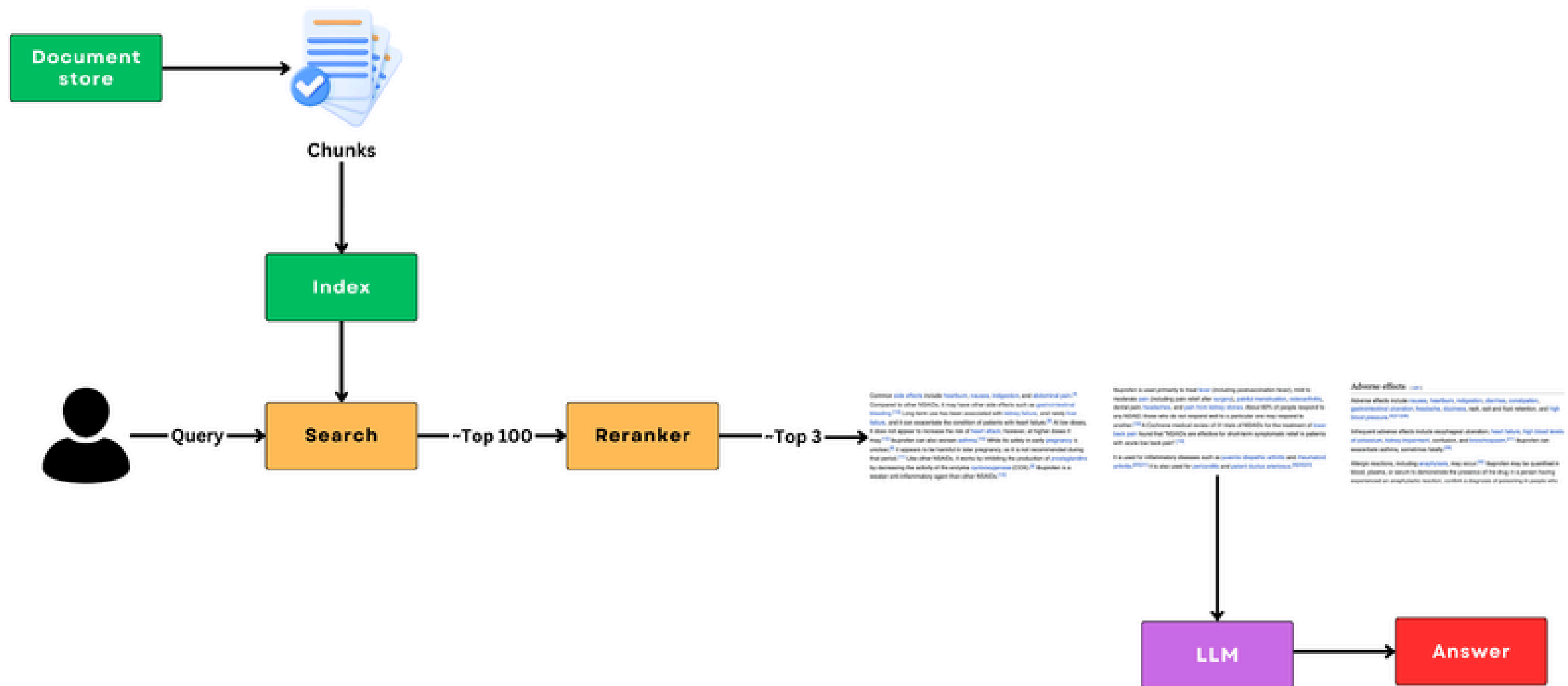
**Gábor Recski**

**Slides by:**

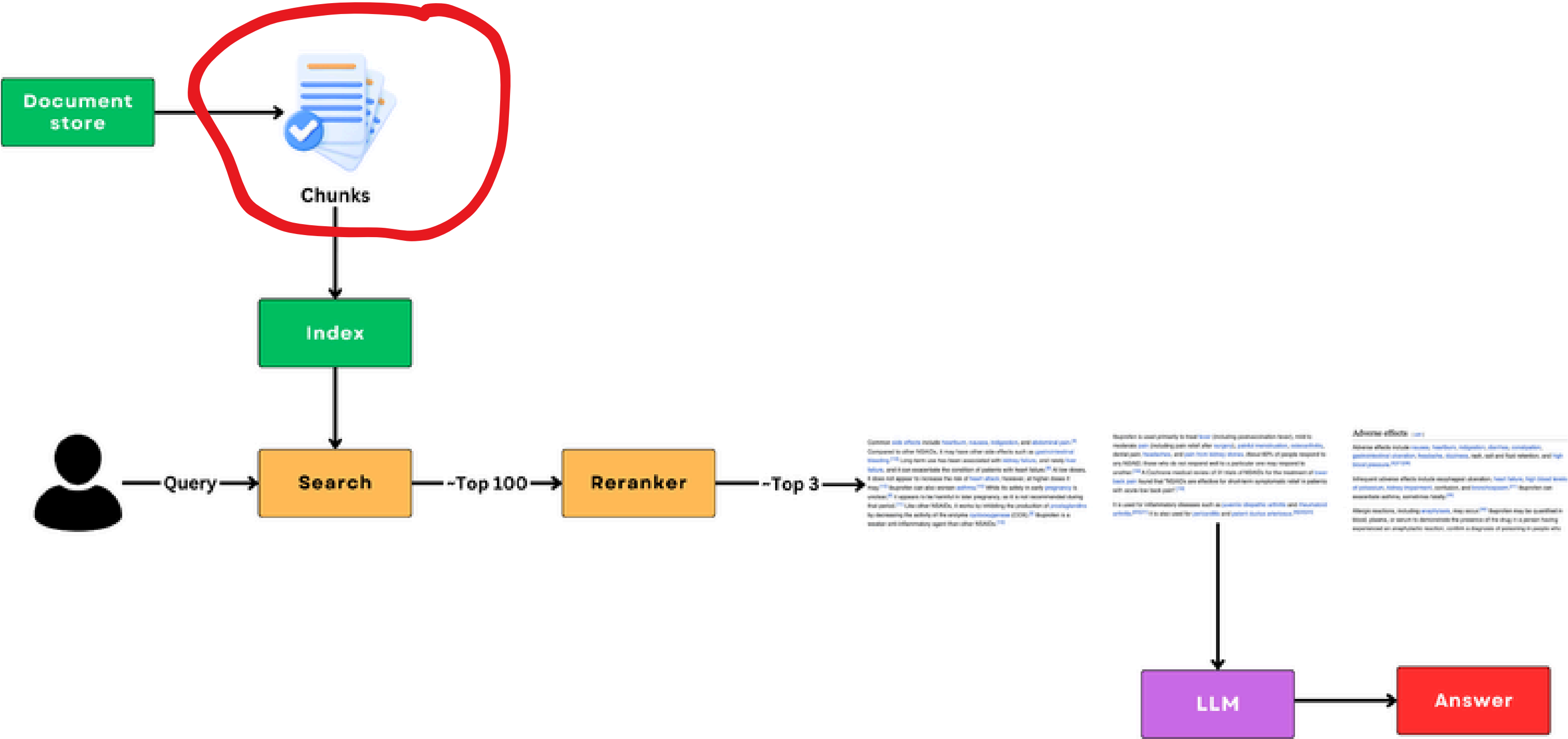
**Ádám Kovács (KR Labs)**

**[kovacs@krlabs.eu](mailto:kovacs@krlabs.eu)**

# Simple RAG



# Chunking



Abstract

We present a lightweight, domain-agnostic verbatim pipeline for evidence-grounded question answering. Our pipeline operates in two steps: first, a sentence-level extractor flags relevant note sentences using either zero-shot LLM prompts or supervised ModernBERT classifiers. Next, an LLM drafts a questionspecific template, which is filled verbatim with sentences from the extraction step. This prevents hallucinations and ensures traceability. In the ArchEHR-QA 2025 shared task, our system scored 42.01%, ranking top-10 in core metrics and outperforming the organiser's 70B-parameter Llama-3.3 baseline. We publicly release our code and inference scripts under an MIT license.

sively with verbatim sentences selected from the extraction phase.

1 Introduction

Modern question-answering (QA) and retrievalaugmented generation (RAG) systems play a vital role in many high-stakes domains for information extraction and generation tasks. In medicine, a typical use case involves clinicians asking questions based on a patient's electronic health record (EHR) notes, rather than manually sifting through lengthy notes, which can be time-consuming. However, in practice, RAG and QA pipelines often misalign evidence and produce incorrect information, commonly referred to as hallucinations (Ji et al., 2023; Madsen et al., 2024). We argue that a reliable QA system should guarantee complete traceability of answers. To tackle this problem, we propose a verbatim pipeline that clearly separates extraction and generation to mitigate hallucinations:

- Sentence-level extraction , using either zeroshot LLMs or supervised ModernBERT classifiers.
- Template-constrained generation , dynamically creating answer templates filled exclu-

We participated in the ArchEHR-QA 2025 shared task on grounded question answering (QA) from electronic health records (EHRs). Our approach involved (i) utilizing a zero-shot gemma-3-27b-it 1 LLM (Team et al., 2025) and (ii) generating synthetic data for sentence extraction from EHRs to train a compact extractor. For this purpose, we trained a Clinical ModernBERT classifier (Lee et al., 2025; Warner et al., 2024), achieving performance comparable to the LLM extractor. Both extractors were then fed into the same LLM template generator. Our solution achieved an overall score of 42.01% , ranking in the top 10 for core metrics, and surpassed the organizers' 70Bparameter Llama-3.3 baseline by a large margin.

Our contributions include a modular, traceable QA architecture that mitigates hallucinations, a method to generate synthetic EHR question-answer corpus and train custom models. Additionally, we are releasing all the code on GitHub 2 under the MIT License. The remainder of the paper discusses background (Section 2), method (Section 3), and evaluation (Section 4).

Abstract

We present a lightweight, domain-agnostic verbatim pipeline for evidence-grounded question answering. Our pipeline operates in two steps: first, a sentence-level extractor flags relevant note sentences using either zero-shot LLM prompts or supervised ModernBERT classifiers. Next, an LLM drafts a questionspecific template, which is filled verbatim with sentences from the extraction step. This prevents hallucinations and ensures traceability. In the ArchEHR-QA 2025 shared task, our system scored 42.01%, ranking top-10 in core metrics and outperforming the organiser's 70B-parameter Llama-3.3 baseline. We publicly release our code and inference scripts under an MIT license.

sively with verbatim sentences selected from the extraction phase.

1 Introduction

Modern question-answering (QA) and retrievalaugmented generation (RAG) systems play a vital role in many high-stakes domains for information extraction and generation tasks. In medicine, a typical use case involves clinicians asking questions based on a patient's electronic health record (EHR) notes, rather than manually sifting through lengthy notes, which can be time-consuming. However, in practice, RAG and QA pipelines often misalign evidence and produce incorrect information, commonly referred to as hallucinations (Ji et al., 2023; Madsen et al., 2024). We argue that a reliable QA system should guarantee complete traceability of answers. To tackle this problem, we propose a verbatim pipeline that clearly separates extraction and generation to mitigate hallucinations:

- Sentence-level extraction , using either zeroshot LLMs or supervised ModernBERT classifiers.
- Template-constrained generation , dynamically creating answer templates filled exclu-

We participated in the ArchEHR-QA 2025 shared task on grounded question answering (QA) from electronic health records (EHRs). Our approach involved (i) utilizing a zero-shot gemma-3-27b-it 1 LLM (Team et al., 2025) and (ii) generating synthetic data for sentence extraction from EHRs to train a compact extractor. For this purpose, we trained a Clinical ModernBERT classifier (Lee et al., 2025; Warner et al., 2024), achieving performance comparable to the LLM extractor. Both extractors were then fed into the same LLM template generator. Our solution achieved an overall score of 42.01% , ranking in the top 10 for core metrics, and surpassed the organizers' 70Bparameter Llama-3.3 baseline by a large margin.

Our contributions include a modular, traceable QA architecture that mitigates hallucinations, a method to generate synthetic EHR question-answer corpus and train custom models. Additionally, we are releasing all the code on GitHub 2 under the MIT License. The remainder of the paper discusses background (Section 2), method (Section 3), and evaluation (Section 4).



# Chunking

- Usually the document store contains lots of **long documents**, e.g. wikipedia articles (we need to link the answer to these documents)
- But we need to split the documents into smaller pieces that we want to index
- **Why?**
  - ML models still **can't process** long documents well (Embeddings, Reranker, LLM, etc..)
  - Smaller size usually means better precision, but less context
  - Needs a good balance

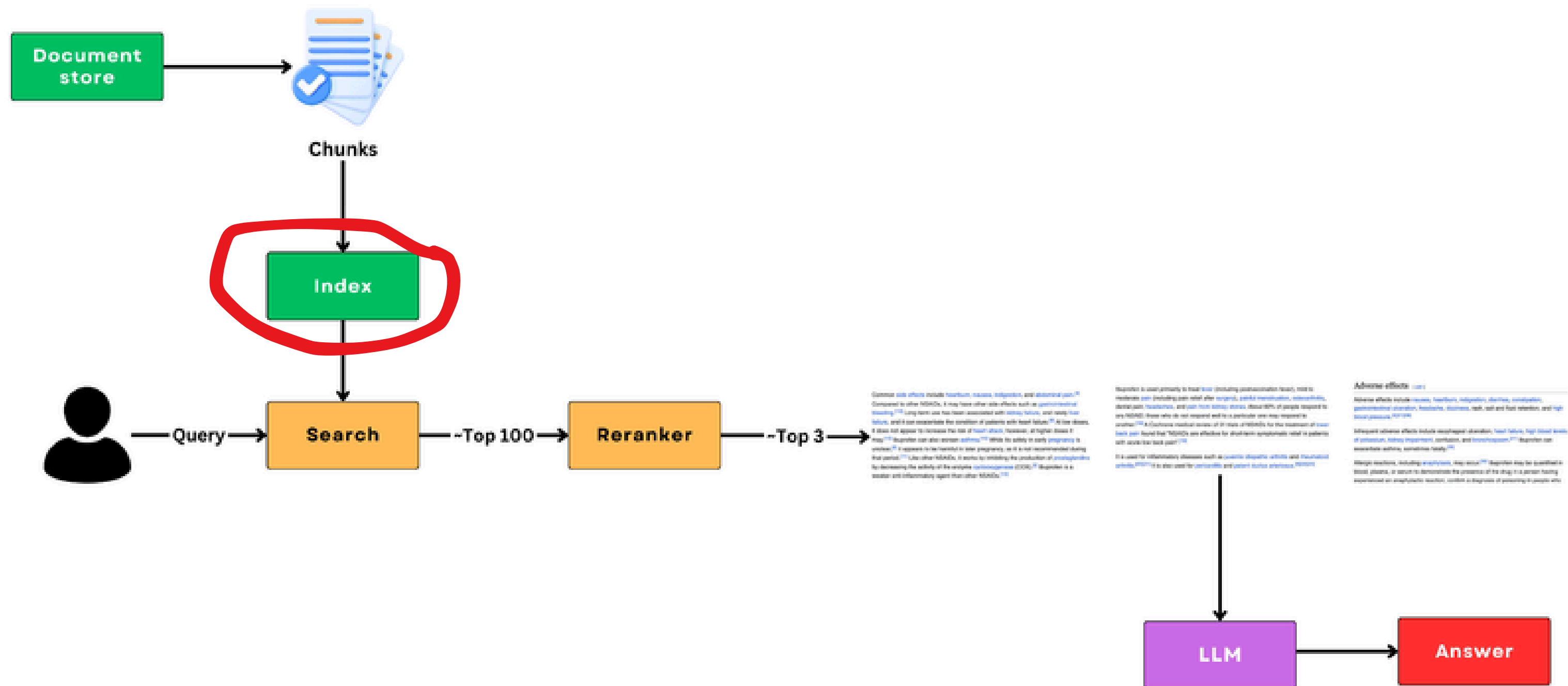


# Chunking

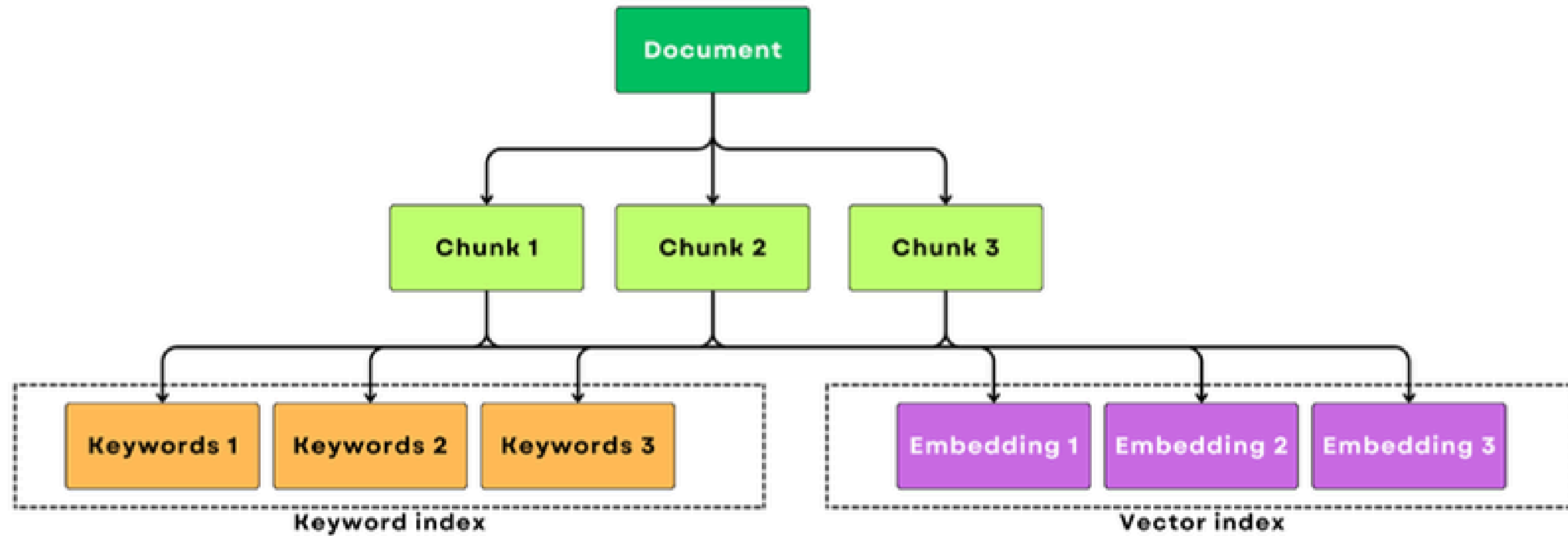
- **What is the *correct* size?**
  - Hard to set, up for experimentation
  - Most ML models still have around 512 token limit (8k for ModernBERT)
- **How to split?**
  - Based on structure (XML, JSON)
  - NLP methods (tokenizer, sentencizer, etc..)
  - Sliding window approach
  - Semantic chunking?
  - spaCy, NLTK, etc..



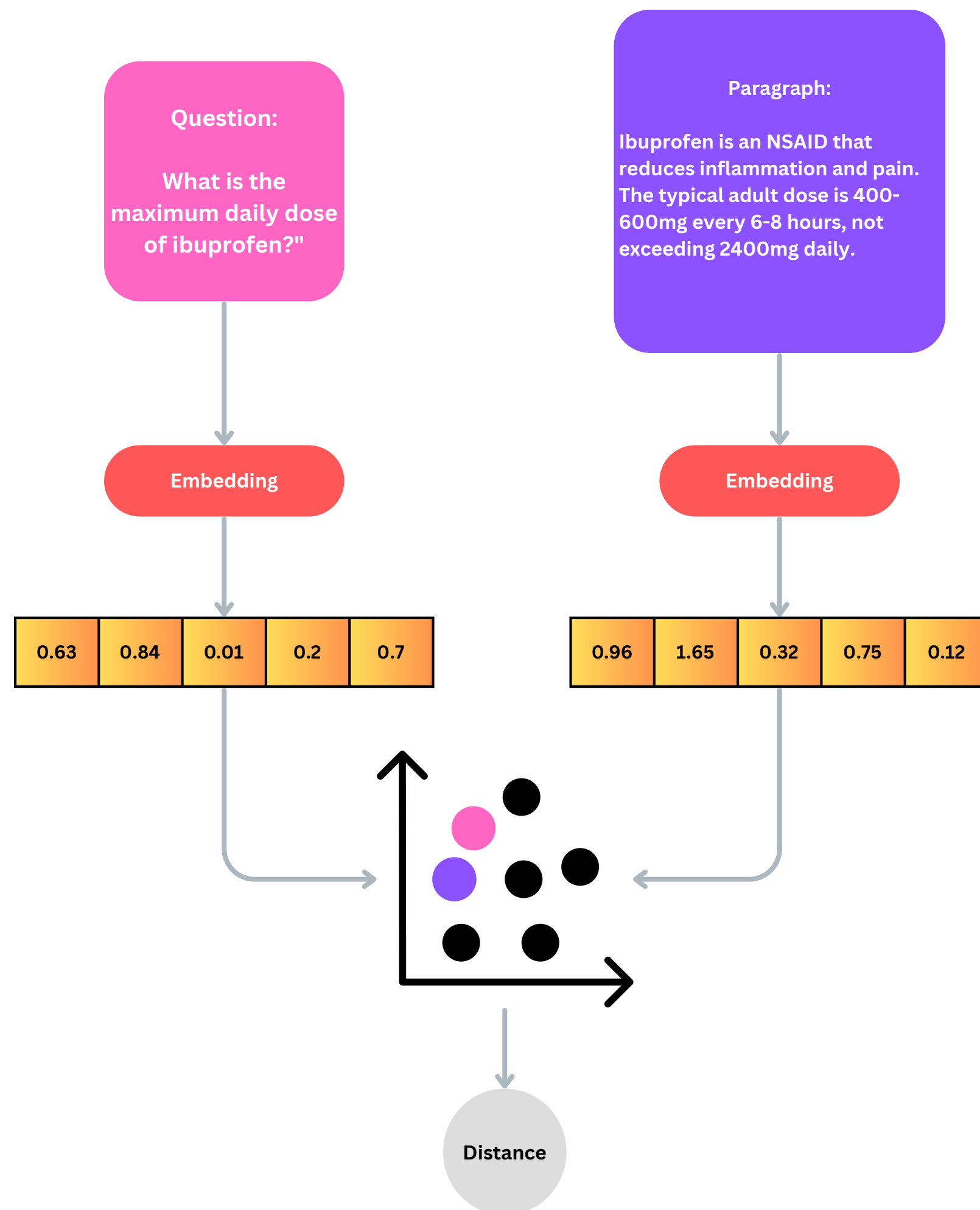
# Indexing



# Indexing







# Indexing - Vectors

- Turn chunks into embeddings
- Use **sentence-embeddings**
- Queries to **relevant** passages are “more” similar than to **not-relevant** passages

# Indexing - Vectors

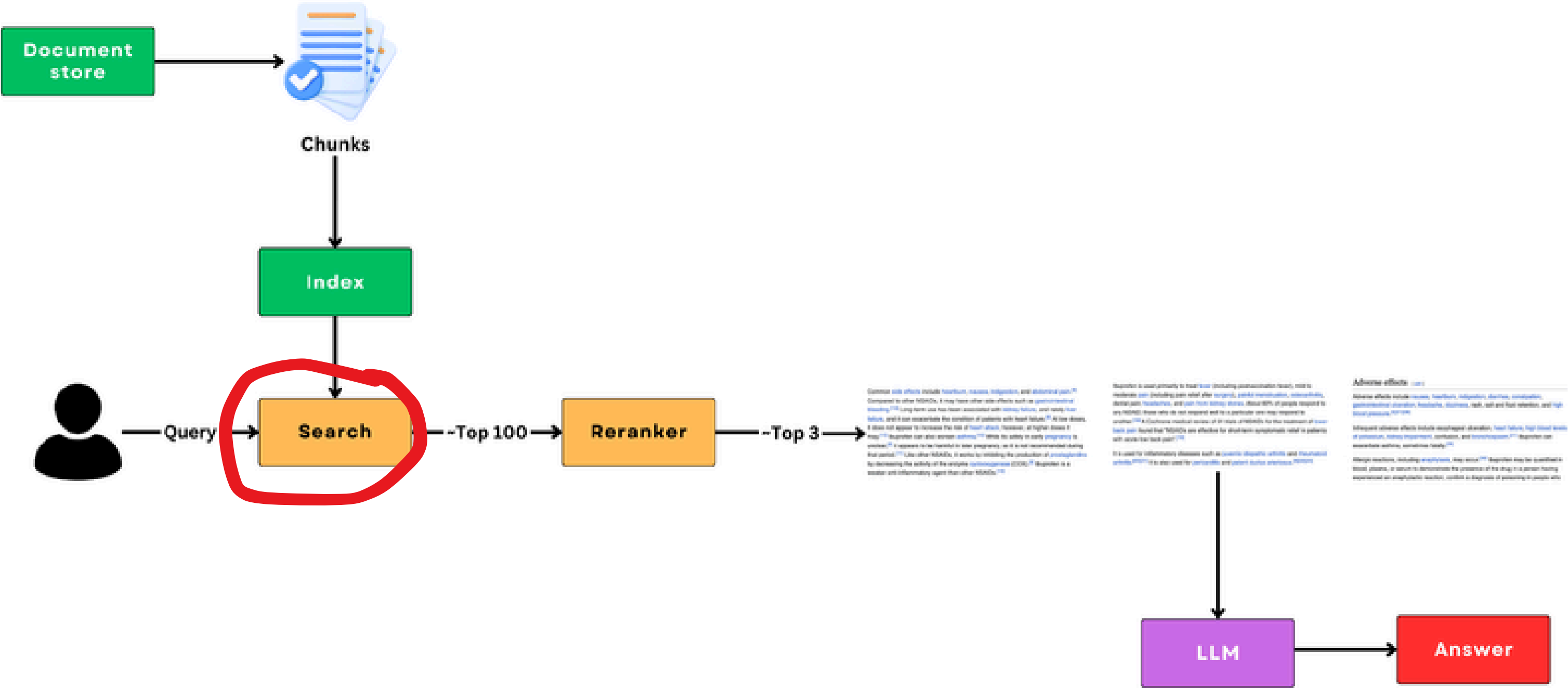
- The embeddings of the chunks are stored in a vector index
- **In memory** index vs. vector **database**
- The functionality we need: fast KNN with vectors
- Voyager -> memory vector store
- FAISS -> memory vector store
- Weaviate -> fully fledged vector database
- Elasticsearch -> also has vector database capability

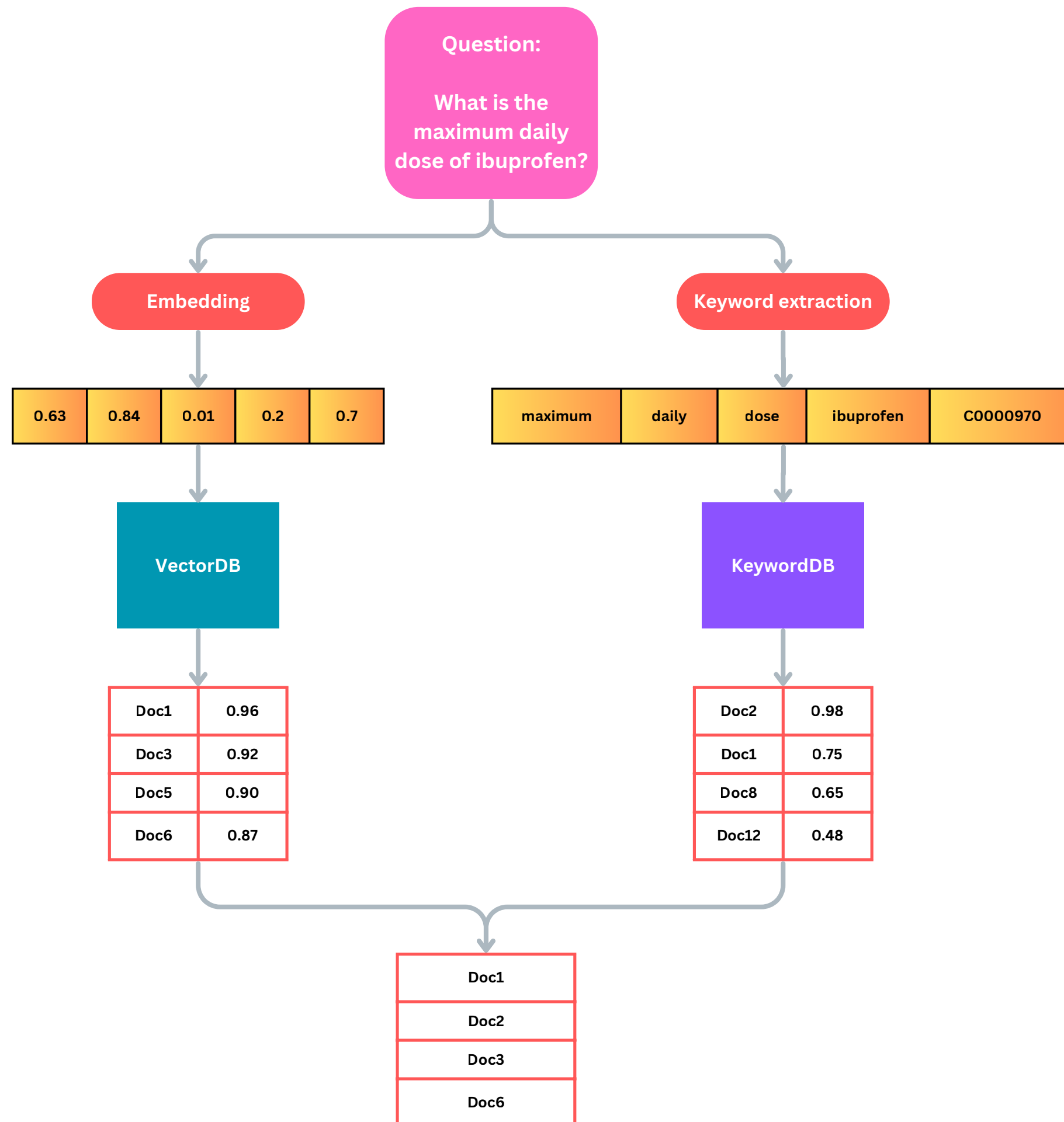
Vector search needs to compute KNN efficiently, comparing to every vector is costly -> ANN (approximate nearest neighbors) computes similarity to clusters instead

# Indexing - Full text search

- Transform chunks into list of tokens (keywords)
- Store the keywords in an inverted index
- Text -> Keywords (*tokenization, lemmatization, filtering stopwords, synonyms, etc..*)
- **Elasticsearch**
  - Go to solution for full text search
  - Rich feature set with language analyzers, synonym lists, etc..
  - Scaling
  - Fuzzy search
- **Tantivy**
  - **Fully** open source
  - Works locally without setting up a server

# Search



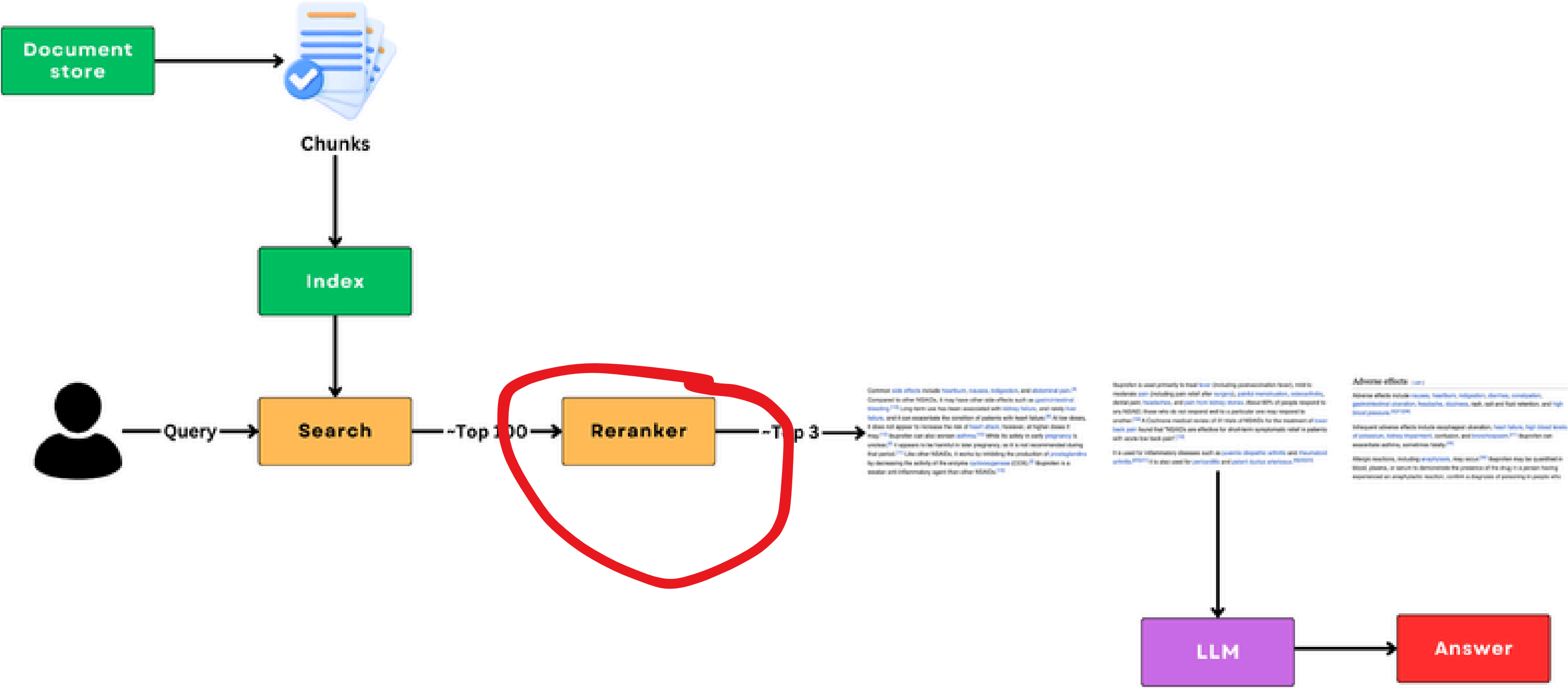


# First stage search

- ~100 relevant candidates
- Hybrid model
- Question preprocessing
- Needs to be **fast**
- **VectorDB** → Cosine similarity
- **KeywordDB** → BM25
- Result Fusion

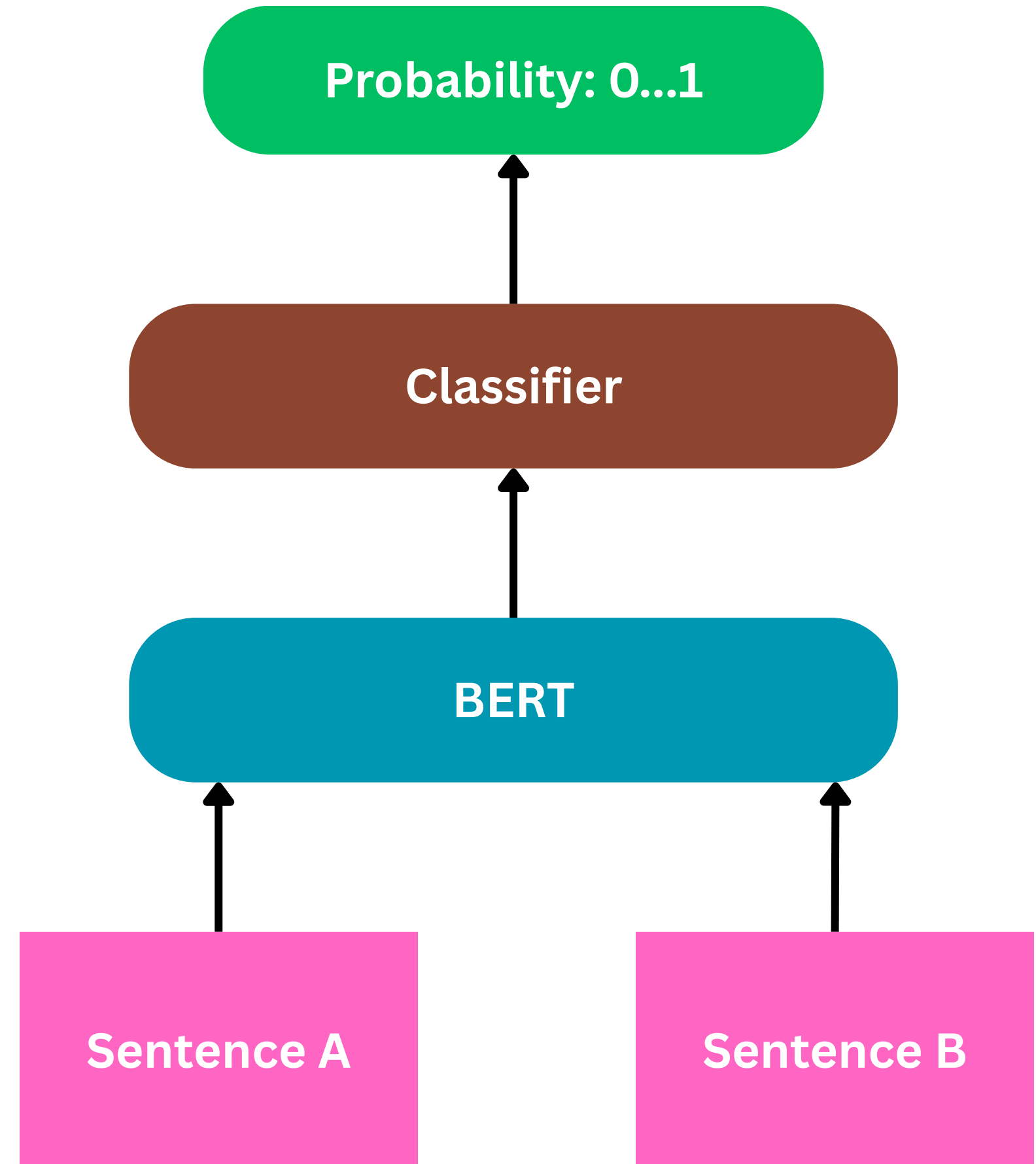


# Reranker



# Reranker

- From **top 100** to **top ~5**
- Much more **expensive**  
(classification of each pairs)
- The task is usually solved with CrossEncoders





# Indexing - Vectors

- Big, public datasets: MSMARCO
- Pick a good model trained on these datasets, e.g. MiniLM
- Collect training data on your field
- **Query-Passage-Relevancy** triplets
- **Finetune** on your domain as a binary classification problem
- (there are also other way to model this task)

# Query: ibuprofen dosage for kids

## Dosage and how often to give it

You'll usually give your child ibuprofen 3 or 4 times a day. Your pharmacist or doctor will tell you how often to give it.

If you're not sure how much to give a child, ask your pharmacist or doctor.

If you give it:

- 3 times in 24 hours, leave at least 6 hours between doses
- 4 times in 24 hours, leave at least 4 hours between doses



# Query: ibuprofen dosage for kids

## Ibuprofen and breastfeeding

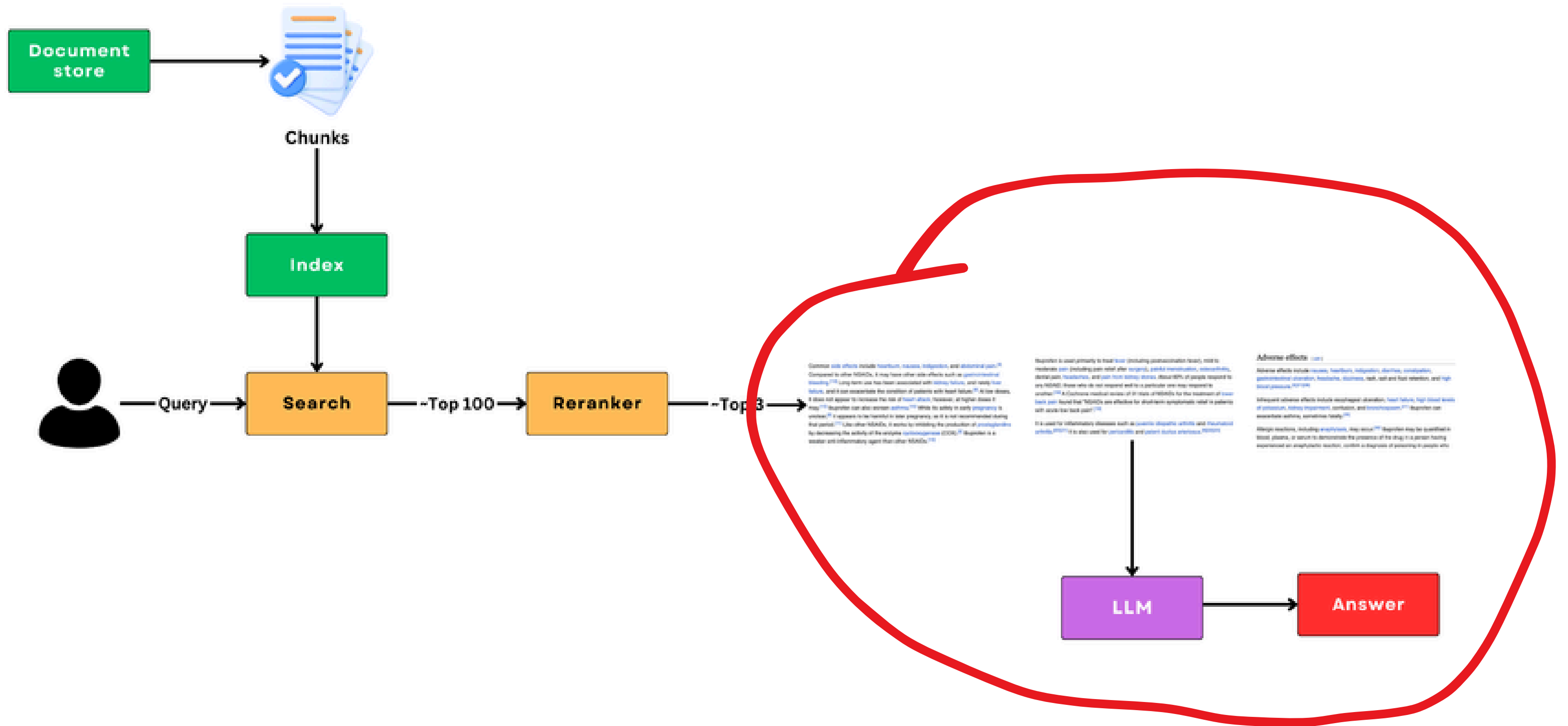
You can take ibuprofen or use it on your skin while breastfeeding. It is one of the painkillers that's usually recommended if you're breastfeeding.

Only tiny amounts get into breast milk and it's unlikely to cause side effects in your baby. Many people have used it while breastfeeding without any problems.

If you notice that your baby is not feeding as well as usual, or if you have any other concerns about your baby, talk to your midwife, health visitor, pharmacist or doctor as soon as possible.



# Generation



# Generation

- The last step is to take the output of the reranker (~top 3 chunks), form it as a *context* for the LLM and prompt it to answer based on the documents
- **Pros:**
  - We can make the LLM to answer based on our own data without actually training its internal knowledge
  - We can fact-check the given answers if the context serves as something that contains the gold answer for the query
  - We can reduce hallucinations (**How?**)



# Forming the prompt

**Prompt:**

Answer questions based on medical documents.

**Question:** What are common symptoms of a peanut allergy?

**First Document:**

*Peanut Allergy*

*Symptoms*

*Peanut allergies can cause mild to severe reactions. Common symptoms include itching, hives, swelling, and difficulty breathing. In severe cases, anaphylaxis can occur, which requires immediate medical attention.*

**Second Document:**

*Allergies and Immune Response*

*When someone with a peanut allergy consumes peanuts, the body's immune system reacts abnormally by releasing chemicals like histamine. This can result in symptoms such as skin rashes, gastrointestinal issues (nausea, vomiting), or respiratory distress.*

**Your answer:**

**Common ...**

# Forming the prompt

**Prompt:**

Answer questions based on medical documents.

**Question:** What are common symptoms of a peanut allergy?

**First Document:**

*Peanut Allergy*

*Symptoms*

*Peanut allergies can cause mild to severe reactions. Common symptoms include itching, hives, swelling, and difficulty breathing. In severe cases, anaphylaxis can occur, which requires immediate medical attention.*

**Second Document:**

*Allergies and Immune Response*

*When someone with a peanut allergy consumes peanuts, the body's immune system reacts abnormally by releasing chemicals like histamine. This can result in symptoms such as skin rashes, gastrointestinal issues (nausea, vomiting), or respiratory distress.*

**Your answer:**

**Common symptoms ...**

# Forming the prompt

**Prompt:**

Answer questions based on medical documents.

**Question:** What are common symptoms of a peanut allergy?

**First Document:**

*Peanut Allergy*

*Symptoms*

*Peanut allergies can cause mild to severe reactions. Common symptoms include itching, hives, swelling, and difficulty breathing. In severe cases, anaphylaxis can occur, which requires immediate medical attention.*

**Second Document:**

*Allergies and Immune Response*

*When someone with a peanut allergy consumes peanuts, the body's immune system reacts abnormally by releasing chemicals like histamine. This can result in symptoms such as skin rashes, gastrointestinal issues (nausea, vomiting), or respiratory distress.*

**Your answer:**

**Common symptoms include ...**



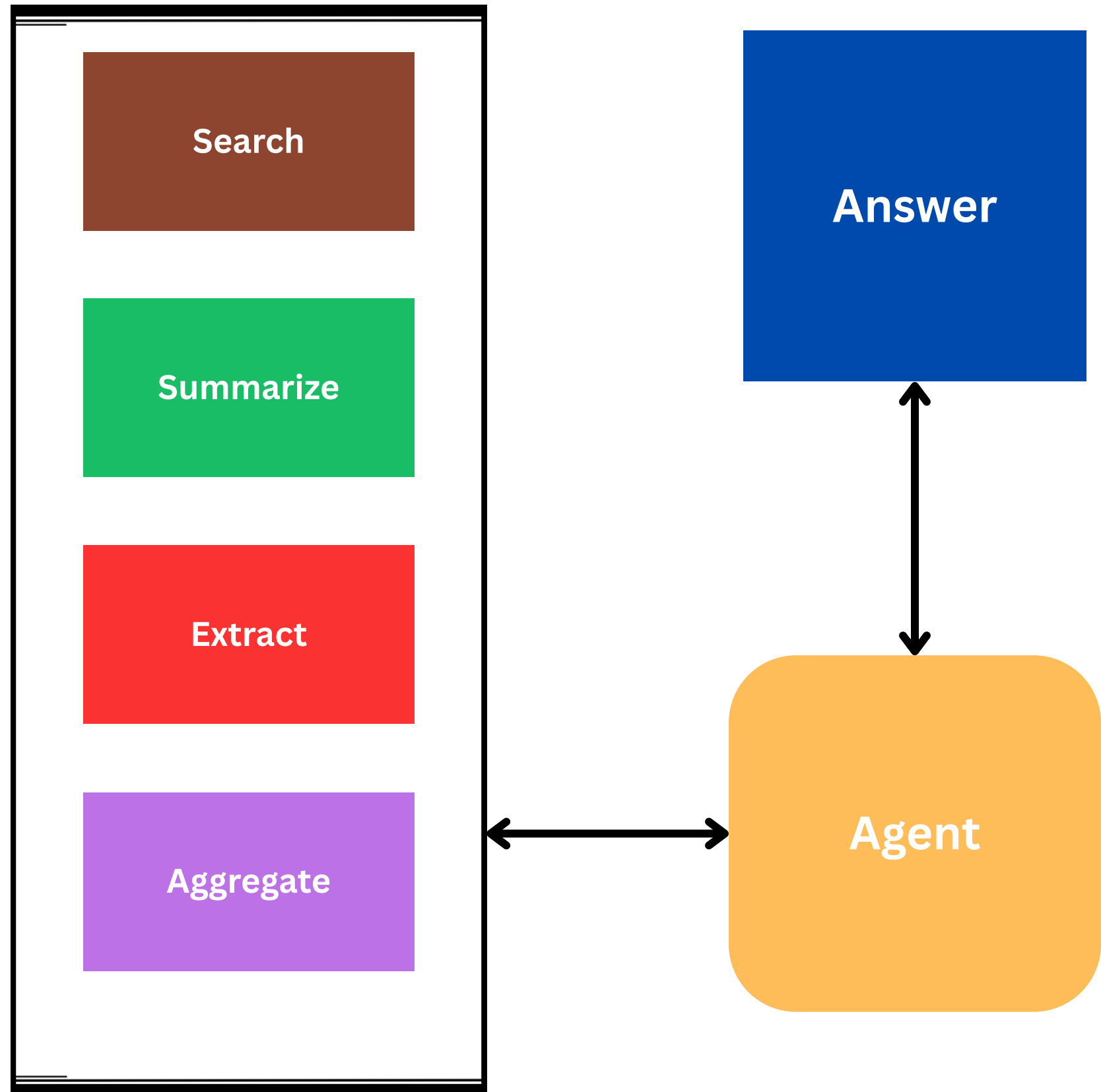
# Generation

- LLMs are just ML models that take sequence of words as an input and generate the next one in an autoregressive manner
- To handle efficient requests, usually there is a need for an optimized LLM server architecture
  - This can be an API, like OpenAI
  - Or you can run your own LLM servers
  - Suggestion, use VLLM as your local server
  - VLLM runs a REST API with the same interface as OpenAI

# Generation

- There are good libraries to orchestrate LLM apps, e.g.
  - LangChain
  - Llamaindex
  - Haystack
- These libraries let you build quick LLM solutions, but hide lots of details from you
- The suggestion is to try to construct your own prompts and only use LLM REST APIs to handle the rest, this way you will have control over your solution

# Tools



## Agentic RAG

- LLMs they act as agents.
- Agents can decide which tools to use
- They can plan multi-step workflows
- They can critique and refine their own outputs
- They adapt dynamically instead of following a fixed pipeline.
- **Reliability?**

# Trustworthy RAG

- **Detect hallucinations** in your RAG
  - LettuceDetect
- What if we don't let LLMs **freely generate**
  - Verbatim-RAG
  - Extract **exact spans** → hallucination free RAG

